

ChargeOps

Milestone 1 Local Application Report

EV Charging Network Operations Platform

Team: [Team Member Name(s)]

Course / Section: [Course / Section]

Submission date: [Submission Date]

MILESTONE OUTCOME ChargeOps is a fully functional local application with an integrated React front end, Node.js/Express application layer, MySQL relational store, local object storage, and background job workers.

Executive summary

ChargeOps gives an internal EV charging operations team one place to monitor infrastructure, coordinate field maintenance, review charging activity, and reconcile payments. The Milestone 1 implementation demonstrates the complete user path from a browser action to application logic and persistent data. It also establishes local equivalents of cloud-oriented capabilities—object storage, queues, workers, telemetry ingestion, and autoscaling supervision—without claiming that the system has already migrated to the cloud.

1. Business domain and project vision

Problem statement

An EV charging network is both a physical service network and a transaction platform. Operators must understand charger availability, associate incidents with the correct equipment and station, dispatch technicians, confirm that completed sessions are billed once, and retain evidence such as field photos and service reports. When these activities are isolated in separate tools, incident response slows down and operational data becomes difficult to reconcile.

Target users

Role	Primary responsibility	Typical actions
Operations Manager	Network-wide coordination	Monitor KPIs; assign technicians; start or resolve work; investigate sessions and payments
Field Technician	On-site service execution	Review assigned work; submit photos/PDF reports; update service progress
Read-only Viewer	Operational visibility	Inspect dashboards and records without changing production state

Why this domain

The domain was selected because it supports a clear core workflow while naturally exercising three cloud-computing dimensions. Relational integrity is required across users, chargers, sessions, payments, wallets, and maintenance records. Unstructured photos, PDF service reports, and invoice files require durable object handling. Telemetry, uploads, billing, and end-of-session events benefit from asynchronous processing when traffic arrives in bursts.

Project vision

VISION Give the operations team a trustworthy path from network condition to field action to financial outcome, with every important change traceable through the application and data layers.

2. Core user flow

The primary Milestone 1 demonstration follows an operations manager from authentication through incident resolution and billing review. Each step exercises the front end, API/application logic, and persistent store.

1. Sign in. The user authenticates through the API and receives permissions associated with an operations role.
2. Review network health. The dashboard requests aggregated counts, charger availability, revenue by city, and recent operational activity.
3. Inspect assets. Station and charger views expose the relational hierarchy connecting locations, equipment, sessions, and incidents.
4. Dispatch maintenance. The manager assigns a technician, starts work, and resolves the incident; the application validates each transition and persists status and timestamps.
5. Trace charging revenue. The manager connects a completed charging session to payment status, wallet movement, invoice metadata, and the invoice file.
6. Process field evidence. A technician uploads a photo or PDF; the API stores the file, queues processing, and a worker classifies the report and creates or updates the maintenance record.
7. Observe asynchronous health. Cloud ops shows queue depth, worker activity, retry/dead-letter behavior, and locally supervised capacity.

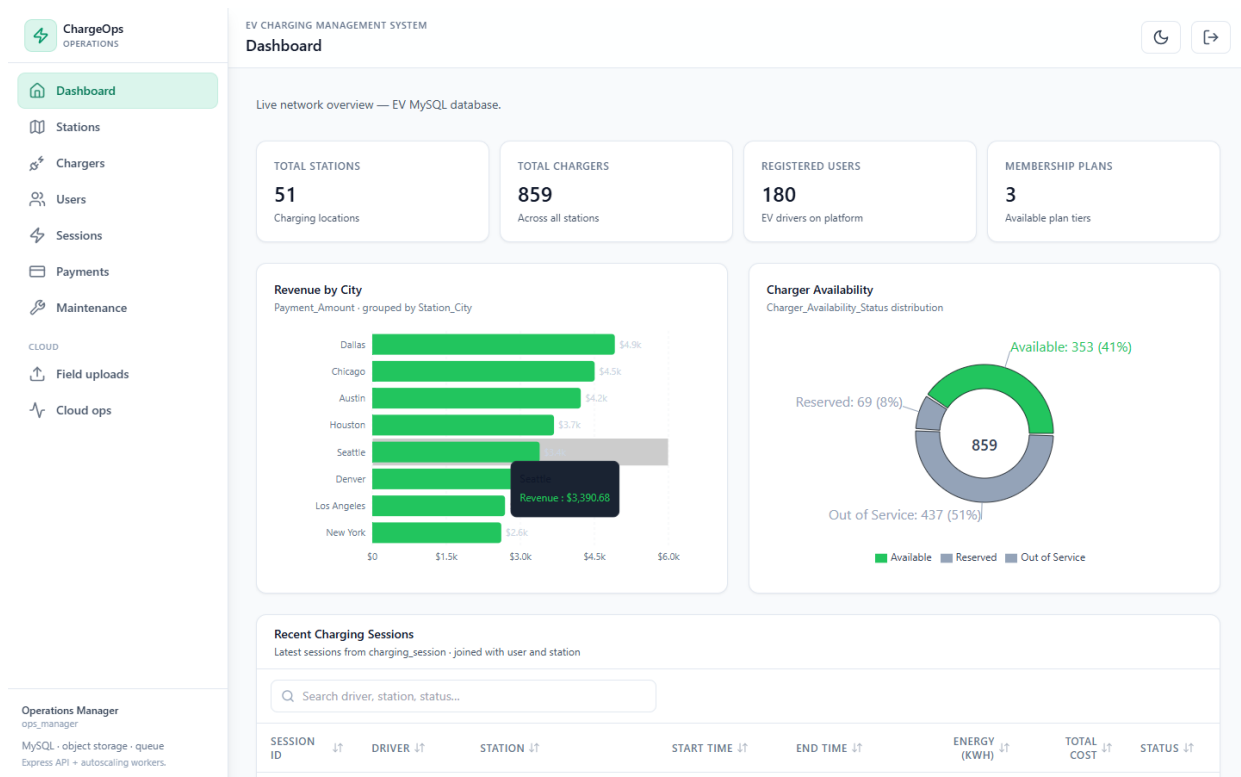


Figure 1. ChargeOps local dashboard populated from the application API and MySQL data.

3. Local architecture

ChargeOps uses a layered architecture. The components run together on the local machine but communicate through stable interfaces that can be replaced with managed cloud services later.

Component	Milestone 1 implementation	Responsibility
Front end	React 18, Vite, Tailwind CSS, Recharts, Leaflet	Role-based operations interface; dashboards, maps, searchable tables, and workflow controls
Application layer	Node.js, Express, REST API, authentication/RBAC, queue adapters, workers	Validate requests; enforce business rules; orchestrate billing, uploads, telemetry, and maintenance
Data store	MySQL plus local object storage	Persist relational records, job state, telemetry metadata, photos, PDFs, and generated invoices

Request path

A browser action calls an authenticated API endpoint. The Express layer validates the user's role and the requested state transition. Short operations read or write MySQL directly within the request. Longer operations—such as billing or file parsing—create a durable job and return an accepted response. A background worker claims the job, performs the work, updates persistent status, and makes the result available to the front end.

Relational model

The principal entities are company, user, company membership, technician, station, charger, wallet, subscription, charging session, payment, and maintenance log. Their relationships allow the application to trace one operational event across physical infrastructure, user activity, and financial records.

Authorization boundary

The front end may hide unavailable actions for usability, but the API remains the authoritative security boundary. It checks authentication and role permissions before allowing assignments, status changes, billing operations, or administrative access.

4. Technical dimensions

Dimension	ChargeOps evidence	Why it matters for later cloud work
Relational data	Stations, chargers, users, sessions, payments, wallets, subscriptions, maintenance, foreign keys, and transactional billing	Managed relational storage, backups, constraints, indexing, and transaction reliability
Unstructured files	Technician JPEG/PDF uploads and generated invoice PDFs in local object storage	Durable object storage, lifecycle rules, controlled access, and event-driven processing
Async / traffic spikes	Separate queues for billing, files, telemetry, and simulation; worker pool; retries, backoff, dead-letter handling, and idempotency	Decoupled managed queues and horizontally scalable consumers during bursty demand

Unstructured-file workflow

When a technician uploads a field photo or PDF, the API writes the binary file to local object storage and records metadata. A file worker parses image metadata or PDF text, classifies fault type, severity, and error code, and can compare GPS information with known station locations. The result is linked to a maintenance ticket, turning field evidence into an operational action.

Asynchronous reliability

Queue jobs carry persistent status so a transient process restart does not erase the work. Retry delays and dead-letter state make failures visible instead of creating an endless loop. Idempotency prevents a repeated request from generating duplicate payments, invoice records, or simulation runs. The API can acknowledge accepted work quickly while workers complete the expensive step independently.

Cloud migration seams

The current local design does not depend on a specific cloud provider. Conceptually, the static web client can move behind a content-delivery service; the Express API and workers can run as containers; MySQL can move to a managed relational database; object files can move to object storage; and MySQL-backed queues can be replaced with a managed queue. These are future mappings, not requirements for Milestone 1.

5. Operational realism and data integrity

Daily simulation

A scheduled MySQL event enqueues one bounded simulation per date. The worker creates realistic platform activity: charging sessions, charger telemetry, payments, invoice files, and maintenance situations. A run typically produces twelve to twenty-four sessions and one to three maintenance situations, then resolves a subset of older simulated incidents to keep the environment active without unbounded growth.

The verified August 31, 2026 run produced 18 sessions and 2 situations, totaling 1,807.80 kWh and \$792.96. All 18 payments and invoice records completed successfully. Integrity checks found no negative wallets and no duplicate billing.

Data repair completed before submission

- Released 250 charger records left in a stuck operational state.
- Expired 109 stale subscriptions.
- Cancelled 53 completed sessions that had no successful payment.
- Marked one abandoned pending payment as failed.
- Restored invoice metadata for 10 payments where the file record was missing.

A backup was retained before repair. These corrections make the demo data internally consistent and allow the video to explain billing and maintenance behavior without presenting known anomalies as normal operation.

Verification evidence

Check	Result	What it demonstrates
Production front-end build	Passed	The React application compiles for deployment.
Application self-test	26 passed	Core API, data, role, and worker behaviors satisfy automated checks.
End-to-end smoke test	18 passed	The local front end, application layer, and data store operate together.
Simulation integrity	Passed	No negative wallets or duplicate billing in the verified run.

6. Milestone 1 requirement alignment

Requirement / rubric area	ChargeOps submission evidence
Working demo — functionality	Login, dashboard, station/charger inspection, maintenance transitions, sessions/payments, file uploads, and queue monitoring run locally end to end.
Domain fit and technical justification	EV charging operations has a clear internal user, business workflow, relational model, unstructured files, and bursty asynchronous workloads.
Video demo quality	The companion script allocates ten minutes, includes camera cues, and walks through a coherent core user flow instead of disconnected screens.
PPT clarity — maximum two slides	Slide 1 states the name, problem, users, team placeholders, and domain rationale. Slide 2 shows the three-layer architecture and technical dimensions.
Team collaboration evidence	Replace the placeholders below with team roles, commits, meeting notes, or a shared task record before submission.

Team collaboration record

Team member	Contribution	Evidence to attach or reference
[Team Member Name]	[Front end / application / database / testing / presentation]	[Commit, pull request, task board item, or meeting note]
[Team Member Name]	[Contribution]	[Evidence]
[Team Member Name]	[Contribution]	[Evidence]

Conclusion

ChargeOps meets the Milestone 1 objective by demonstrating a viable domain and a functioning local front end, application layer, and data store. The project's strongest evidence is the connected workflow: operational records are not isolated screens, but traceable data that moves from charger activity and field evidence through validated application logic to maintenance and financial outcomes.

BEFORE SUBMITTING Replace all bracketed placeholders, confirm the final demo account and sample record, keep presenter cameras on, and submit this report together with the two-slide deck and video.